NEURAFLOWW QUANTITATIVE SYSTEMS MASTER ARCHITECTURE, CODEBASE & ENGINEERING HANDOVER MANUAL

Publication-Grade Knowledge Transfer | Transition Notice: Leaving the Company Standard | Date: October 16, 2026

Platform Identity: Neurafloww (Flow Tester Pro) | **Hosting:** Vercel Global Edge CDN | **Database:** Supabase & ClickHouse
Primary Git Remote: https://github.com/cobaltitservices/Neurafloww.git (Branch: main)
Architecture Scale: 18 Cryptocurrencies × 16 Timeframes × 6 Neural Architectures = 1,824 - 2,400 Model Permutations
Core Technologies: Python 3.10+, PyTorch 2.1, ClickHouse, Backblaze B2, AWS EC2/DynamoDB, GCP, Salad Cloud, Vanilla CSS/JS

1. EXECUTIVE SUMMARY & PLATFORM MISSION

Neurafloww is an institutional-grade algorithmic cryptocurrency training platform, vectorized backtesting matrix, and real-time paper trading terminal. Designed to systematically eliminate emotional bias from digital asset execution, the engine orchestrates 6 deep neural network families across 18 liquid cryptocurrency assets and 16 timeframe resolutions. The system evaluates thousands of model iterations continuously, automatically deploying the top 15/24 ranking models into live parallel paper trading workers with institutional order-book slippage and fee simulation.

2. COMPLETE PROJECT HISTORY & ARCHITECTURAL EVOLUTION

Phase & Title	Historical Milestone, Problem Encountered & Architectural Resolution
`Phase 1: Genesis`	Engineered data collectors for 18 crypto assets and 16 timeframes. Integrated ClickHouse (34.132.233.162:8123) for columnar time-series storage. Built Geometric Brownian Motion (GBM) offline fallback engine.
`Phase 2: Model Zoo`	Constructed 6 PyTorch deep learning models (LSTM, Residual GRU, TCN, Transformer, CNN-LSTM, MLP-Mixer). Developed zero-leakage chronological 70/15/15 train/val/test splitters with fit-on-train scalers.
`Phase 3: Multi-Cloud`	Provisioned AWS EC2 (c5.9xlarge, c7i.16xlarge, g5.xlarge), DynamoDB tables, GCP VM (34.170.186.204), and Backblaze B2 cold storage. Integrated Salad Cloud distributed RTX 3090/4090 GPU nodes at 85% cost savings.
`Phase 4: Bloomberg UI`	Crafted cyber-slate dark mode terminal (dashboard/index.html) with TradingView candlestick order flow, marquee ticker tape, dynamic equity curve SVG charts, and Top 15 live worker table.
`Phase 5: Serverless Migration`	Problem: Legacy GCP VM suffered from single-point-of-failure risks, SSH port 22 timeouts, and $200-$400/mo idle costs. Resolution: Decommissioned VM 34.170.186.204. Migrated to GitHub (cobaltitservices/Neurafloww) on Vercel Edge Serverless.
`Phase 6: Mobile & UX`	Standardized navigation labels (Model Matrix, Leaderboard, Paper Trading, Logs, Admin, API). Re-engineered footer columns into scrollable quick-menus. Optimized mobile layout with collapsible drawer.
`Phase 7: Telemetry & UTM`	Built password visibility toggles, UTM traffic attribution tracking with URL builder, 3px glowing scroll depth progress bar, universal command palette (Cmd+K), and sticky headers.
`Phase 8: Rebranding`	Uniform rebranding from Flow Tester to Neurafloww (NEURAFLOWW v3.0 PRO) across all UI, metadata, and manifests. Added localStorage client-side authentication fallback (admin/admin).
`Phase 9: Master Handover`	Created exhaustive Master Handover Documentation (HANDOVER_DOCUMENTATION.md) and revamped Admin Development Diary with glassmorphism cards and live search.

3. BINANCE DATA PIPELINE & FEATURE ENGINEERING

18 Monitored Cryptocurrencies: BTC_USDT, ETH_USDT, SOL_USDT, BNB_USDT, XRP_USDT, ADA_USDT, AVAX_USDT, DOGE_USDT, DOT_USDT, LINK_USDT, NEAR_USDT, SUI_USDT, PEPE_USDT, SHIB_USDT, FLOKI_USDT, BONK_USDT, LTC_USDT, ATOM_USDT.
16 Timeframe Resolutions: 1s, 1m, 3m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 8h, 12h, 1d, 3d, 1w, 1M.
13 Features: OHLCV, 10-period Fast MA, 30-period Slow MA, MA Crossover delta, Bollinger Bands (Upper, Lower, %B, Bandwidth), Parabolic SAR.

Binance Batch Price Ingestion Code (with 1.0s TTL Caching):

```
# Location: paper_trading/engine.py
import urllib.request, json, time

_binance_price_cache = {}
_binance_price_cache_time = 0.0

def fetch_all_binance_prices():
global _binance_price_cache, _binance_price_cache_time
now = time.time()
if _binance_price_cache and (now - _binance_price_cache_time < 1.0):
return _binance_price_cache

url = "https://api.binance.com/api/v3/ticker/price"
req = urllib.request.Request(url, headers={"User-Agent": "Neurafloww/3.0 QuantTerminal"})
with urllib.request.urlopen(req, timeout=2.5) as resp:
data = json.loads(resp.read().decode())
_binance_price_cache = {item["symbol"]: float(item["price"]) for item in data}
_binance_price_cache_time = now
return _binance_price_cache
```

Code Explanation: Ingests all 2,000+ Binance pairs in 1 batch HTTP request consuming only 2 request weights instead of 18 separate calls, while the 1.0-second in-memory TTL prevents hitting Binance 1,200 weight/minute rate limits during high-frequency paper loops.

4. CLICKHOUSE COLUMNAR ANALYTICS DATABASE

Host: 34.132.233.162:8123 (HTTP) | **Database:** crypto_data | **Table:** ohlcv_data
ClickHouse handles billions of historical candle records with column-oriented compression codecs (DoubleDelta for timestamps, Gorilla and ZSTD for prices), delivering sub-millisecond query execution.

ClickHouse Client & Geometric Brownian Motion Synthetic Generator:

```
# Location: data/clickhouse_client.py
import numpy as np, pandas as pd

class ClickHouseDataClient:
def fetch_data(self, pair: str, timeframe: str, limit: int = 5000):
if self._connected and self.client:
query = f"SELECT timestamp, symbol, timeframe, open, high, low, close, volume FROM {self.config.table} WHERE symbol =
%(pair)s AND timeframe = %(tf)s ORDER BY timestamp ASC LIMIT {limit}"
return self.client.query_df(query, parameters={"pair": pair, "tf": timeframe})
return self.generate_mock_data(pair, timeframe, num_rows=limit)

@staticmethod
def generate_mock_data(pair: str, timeframe: str, num_rows: int = 2000):
# S_t = S_0 * exp((mu - 0.5 * sigma^2) * t + sigma * W_t)
base = 50000.0 if "BTC" in pair else (3000.0 if "ETH" in pair else 100.0)
returns = np.random.normal(0.0001, 0.015, num_rows)
closes = base * np.exp(np.cumsum(returns))
opens = closes * (1 + np.random.normal(0, 0.002, num_rows))
highs = np.maximum(opens, closes) * (1 + np.abs(np.random.normal(0, 0.004, num_rows)))
lows = np.minimum(opens, closes) * (1 - np.abs(np.random.normal(0, 0.004, num_rows)))
return pd.DataFrame({"timestamp": pd.date_range(end=pd.Timestamp.now(), periods=num_rows, freq="1h"),
"symbol": pair, "timeframe": timeframe, "open": opens, "high": highs, "low": lows, "close": closes})
```

Code Explanation: Parameterized SQL queries eliminate injection vectors. If ClickHouse connectivity is unavailable, the client seamlessly invokes generate_mock_data() with volatility drift, preventing test suite and local training crashes.

5. BACKBLAZE B2 & COLD STORAGE LAYER

Endpoint: https://s3.us-west-004.backblazeb2.com | **Cost Advantage:** $0.005/GB/mo (78% savings vs AWS S3)
Backblaze B2 archives cold Snappy-compressed Parquet tick files and mirrors model weight checkpoints (.pt, scaler.pkl).

```
# Location: storage/b2_archival_client.py
import boto3, os

def get_b2_client():
return boto3.client("s3",
endpoint_url=os.getenv("B2_ENDPOINT_URL", "https://s3.us-west-004.backblazeb2.com"),
aws_access_key_id=os.getenv("B2_KEY_ID"),
aws_secret_access_key=os.getenv("B2_APPLICATION_KEY"))

def upload_model_checkpoint(model_path, architecture, filename, bucket="neuraflowww-model-checkpoints"):
s3 = get_b2_client()
s3.upload_file(model_path, bucket, f"{architecture}/{filename}")
```

6. AWS CLOUD INFRASTRUCTURE & DYNAMODB SERVICES

EC2 Instances: c5.9xlarge (36 vCPUs, 72 GB RAM), c7i.16xlarge (64 vCPUs, 128 GB RAM), g5.xlarge (NVIDIA A10G GPU), t3.medium.
DynamoDB Tables: PaperTradingTrades, PaperTradingTicks, PaperTradingSessions, CryptoHistoricalPrices.

```
# Location: data/dynamodb_data_client.py
import boto3
from boto3.dynamodb.conditions import Key

def query_dynamodb_historical(table, pair, timeframe, limit=2000):
resp = table.query(
KeyConditionExpression=Key("symbol_tf").eq(f"{pair}#{timeframe}"),
Limit=limit, ScanIndexForward=True
)
return resp.get("Items", [])
```

7. GCP DECOMMISSIONING & SALAD CLOUD GPU ACCELERATION

GCP Decommissioning: Compute VM 34.170.186.204 was permanently decommissioned due to single-point-of-failure risks and high monthly idle costs ($200-$400/mo). All frontend and API routing was migrated to GitHub & Vercel Edge Serverless.
Salad Cloud GPU Fleet: Distributed container fleet utilizing consumer RTX 3080/3090/4090 GPUs at $0.15-$0.25/hr (85% cheaper than AWS G5). Container defined in Dockerfile.salad and executed via train_on_salad_gcs.py.

8. DEEP LEARNING MODEL ZOO (6 ARCHITECTURES)

Model Identifier	Architecture Class	Specialization & Mathematical Role
`model_1_lstm`	MultiLayer LSTM	2-layer stacked LSTM with 0.2 dropout. Captures persistent macro trends.
`model_2_gru`	Residual GRU	Gated Recurrent Unit with LayerNorm and skip connections for volatile regimes.
`model_3_tcn`	TemporalConvNet (1D)	Dilated causal convolutions providing exponential receptive fields.
`model_4_transformer`	TimeSeries Transformer	Multi-head self-attention (h=4) capturing cross-temporal correlations.
`model_5_cnnlstm`	CNNLSTM Hybrid	1D spatial convolution feature extractor paired with sequential LSTM.
`model_6_mlp_mixer`	MLP-Mixer / N-BEATS	Token-Mixing and Channel-Mixing linear MLPs delivering sub-second inference.

MultiLayer LSTM Source Code:

```
# Location: models/model_1_lstm.py
import torch, torch.nn as nn
from .base_model import BaseModel

class Model1LSTM(BaseModel):
def build_model(self):
self.lstm = nn.LSTM(input_size=self.input_dim, hidden_size=self.hidden_dim,
num_layers=self.num_layers, batch_first=True,
dropout=self.dropout if self.num_layers > 1 else 0.0)
self.fc = nn.Sequential(nn.Linear(self.hidden_dim, self.hidden_dim // 2),
nn.ReLU(), nn.Dropout(self.dropout),
nn.Linear(self.hidden_dim // 2, self.prediction_horizon))

def forward(self, x):
lstm_out, _ = self.lstm(x)
return self.fc(lstm_out[:, -1, :])
```

9. AUTONOMOUS PAPER TRADING EXECUTION ENGINE

Parameters: $10,000 USD virtual equity, 0.04% taker fee deduction, 0.01% simulated slippage penalty, 20% max equity allocation per position, 2.5% hard stop-loss.

```
# Location: paper_trading/engine.py
def open_position(self, side: str, market_price: float, size_usd: float):
fill_price = market_price * (1 + self.slippage) if side == "LONG" else market_price * (1 - self.slippage)
fee = size_usd * self.commission
self.equity -= fee
self.position = PaperPosition(side=side, entry_price=fill_price, position_size_usd=size_usd, entry_time=time.time())

def close_position(self, market_price: float):
fill_price = market_price * (1 - self.slippage) if self.position.side == "LONG" else market_price * (1 + self.slippage)
fee = self.position.position_size_usd * self.commission
ret = (fill_price - self.position.entry_price) / self.position.entry_price if self.position.side == "LONG" else
(self.position.entry_price - fill_price) / self.position.entry_price
pnl = (self.position.position_size_usd * ret) - fee
self.equity += pnl
```

10. STEP-BY-STEP SETUP GUIDES & OPERATOR RUNBOOKS

10.1 Local Development Environment:

```
# 1. Clone repository and set up virtual environment
git clone https://github.com/cobaltitservices/Neurafloww.git && cd Neurafloww
python3 -m venv venv && source venv/bin/activate
pip install --upgrade pip && pip install -r requirements.txt

# 2. Run local development terminal
cd dashboard && python3 -m http.server 3000
# Open http://localhost:3000 in your browser
```

10.2 Production Push & Automated Vercel Deployment:

```
# Deploy latest changes to GitHub & trigger Vercel Edge build in <20s
./deploy_to_github.sh "feat: description of changes"

# Run model training across assets
python3 agent.py train --pairs BTC_USDT ETH_USDT --timeframes 1h 4h --models model_1_lstm model_4_transformer

# Run continuous vectorized backtesting daemon
python3 continuous_backtest_daemon.py

# Launch live parallel paper trading daemon
python3 auto_paper_trading_daemon.py
```

10.3 Admin Credentials & Emergency Access:

- **Default Username:** admin | **Default Password:** admin
- **Persistence:** Passwords modified in the Admin Console persist in browser localStorage.
- **Emergency Reset:** Clear browser cache or edit ADMIN_PASSWORD in api/server.py and redeploy.

11. KEY REPOSITORY ARTIFACTS INVENTORY

File Path	Description & Operational Role	Target Environment
`HANDOVER_DOCUMENTATION.md`	Master institutional architecture handover manual (this document)	Root & Docs
`DEVELOPMENT_DIARY.md`	Complete engineering development diary and changelog	Root & Docs
`dashboard/index.html`	Primary Single Page Application (UI, CSS, and JS)	Vercel Static CDN
`dashboard/404.html`	Branded 404 error template with auto-redirect	Vercel Static CDN
`vercel.json`	Vercel Serverless Edge configuration and rewrites	Vercel Infra
`deploy_to_github.sh`	1-click automated synchronization and deployment tool	Local Workspace
`export_website_package.py`	Bundling utility generating website_production_package.zip	Local Workspace
`data/clickhouse_client.py`	ClickHouse time-series client & GBM mock engine	Core Engine
`data/dynamodb_data_client.py`	AWS DynamoDB time-series and trade storage client	Core Engine
`paper_trading/engine.py`	Top 15/24 Paper Trading execution engine	Core Engine
`config.yaml`	Runtime configuration, pairs, timeframes, and hyperparams	Core Engine